



Practical No 3

Aim

Designing a calculator using Typescript

Apparatus / Software Required

- Visual Studio Code / Any Code Editor
- TypeScript
- Node.js
- Web Browser

Theory

A calculator is one of the simplest applications used to understand how user input, program logic, and output work together. In this practical, TypeScript is used to add type safety and better code organization compared to JavaScript.

The calculator accepts two numbers from the user, performs the selected arithmetic operation, and displays the result dynamically.

Concepts Required

1. TypeScript

TypeScript is a superset of JavaScript developed by Microsoft. It adds **static typing**, making the code easier to write, understand, and debug.

Features

- Static Type Checking
- Object-Oriented Programming Support
- Better Error Detection
- Improved Code Readability
- Easy Maintenance

Example:

```
let num: number = 10;  
let name: string = "Bhaskar";
```

2. Variables and Data Types

Variables store the values entered by the user.

Common data types used:

- number
- string
- boolean



Example:

```
let a: number = 20;  
let b: number = 10;
```

3. Functions

Functions perform a specific task.

The calculator uses functions to perform different arithmetic operations.

Example:

```
function add(a: number, b: number): number {  
    return a + b;  
}
```

4. Arrow Functions

Arrow functions provide a shorter syntax for writing functions.

Example:

```
const subtract = (a: number, b: number): number => {  
    return a - b;  
};
```

5. Operators

Arithmetic operators perform mathematical calculations.

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Example:

```
let sum = a + b;
```

6. Conditional Statements

Conditional statements check special cases such as division by zero.

Example:

```
if (b !== 0) {  
    console.log(a / b);  
} else {  
    console.log("Cannot divide by zero");  
}
```



7. DOM (Document Object Model)

The DOM allows TypeScript to interact with HTML elements.

Using the DOM, we can:

- Read input values
- Display output
- Respond to button clicks

Example:

```
const num1 = document.getElementById("num1") as HTMLInputElement;
```

8. Type Assertion

Type assertion tells TypeScript the exact type of an HTML element.

Example:

```
const input = document.getElementById("num1") as HTMLInputElement;
```

Without type assertion, TypeScript cannot identify the properties of HTML elements.

9. Number Conversion

Values obtained from HTML input fields are strings.

They must be converted into numbers.

Example:

```
let a = Number(num1.value);
```

or

```
let a = parseFloat(num1.value);
```

10. Input Validation

Input validation ensures that users enter valid numbers before performing calculations.

Example:

```
if (isNaN(a) || isNaN(b)) {  
    alert("Please enter valid numbers");  
}
```

Benefits:

- Prevents errors
- Improves reliability
- Enhances user experience



Shri Yashwantrao Bhonsale Education Society's

YASHWANTRAO BHONSALE INSTITUTE OF TECHNOLOGY

(DTE CODE : 3470) (MSBTE Code : 1742)

Approved by AICTE, DTE & Affiliated to Mumbai University & MSBTE Mumbai
(NBA Accredited ME, CE, EE Diploma Programs)

Conclusion:
